fes-f2-gpu: the Rust-side preparation cost of a call, before and after a direct pack

Engineering note, not a result on the index-calculus cost. Two A/B runs: a CPU-only one (host serve worker) and one on a Modal A100. Wall times are a practicality note (AGENTS.md section 6); the counted cost is unchanged. All figures are unverified assertions of the producing session [pcc:unverified].

Question

The 2026-11-13 end-to-end report names the per-call overhead of the `fes-f2-gpu` engine as the next thing to cut: 1.0, 1.2 and 3.6 ms per call at 24, 26 and 30 unknowns against kernel times of about 0.1, 0.2 and 1.3 ms. Where does the Rust side of a call spend its time, and what does it cost to build the system the worker takes?

Method

`FES_GPU_TIMING=1` (this change) prints the per-call milliseconds of each Rust-side phase of the `fes-f2-gpu` solver: form conversion, packing, worker round trip, Gray-rank sort, and CPU verification of every point. `ic bench --sweep` ran the two Koblitz sweeps (`koblitz-n19-gpu.json`, 20 unknowns, 786 solver calls; `koblitz-n23-gpu.json`, 24 unknowns, 1878 solver calls) with the host-emulation serve worker `gpu/fes/fes_solve`, so no GPU is involved and the round trip is the CPU worker, not a GPU. Two binaries of the same source, differing only in the change below: before (`ic-timers-before`, sha256 `a6a31a51...`) and after (`target/release/ic`, sha256 `357d1eab...`). Three runs per arm and instance, arm order alternating per pair (`run_ab.sh`), `--rho-runs 2`. The preparation cost is forms plus pack before, and the direct pack alone after. Host: the authoring container, 4 logical CPUs, isolation unstated.

Change. The form path built one `QuadraticForm` (a `Vec<Vec<bool>>`) per equation, converted a second time when an equation is narrower than the system, then packed all of them into the worker's words. `fes_pack_direct` packs the same words in one pass over the monomials and reports the same two counted charges; it declines anything the form path treats differently, and that path then decides.

Results

instance	before, ms/call	after, ms/call	saved, ms/call	saved
Koblitz n=19, 20 unknowns	0.1695	0.0048	0.1647	97.1%
Koblitz n=23, 24 unknowns	0.2784	0.0074	0.2710	97.3%

Mean of three runs per arm; min to max in the figure. The other Rust-side phases are small: sorting is below 0.001 ms per call and CPU verification 0.0025 ms (n=19) and 0.0060 ms (n=23), so the form conversion was about 90 percent of the Rust-side time outside the round trip.

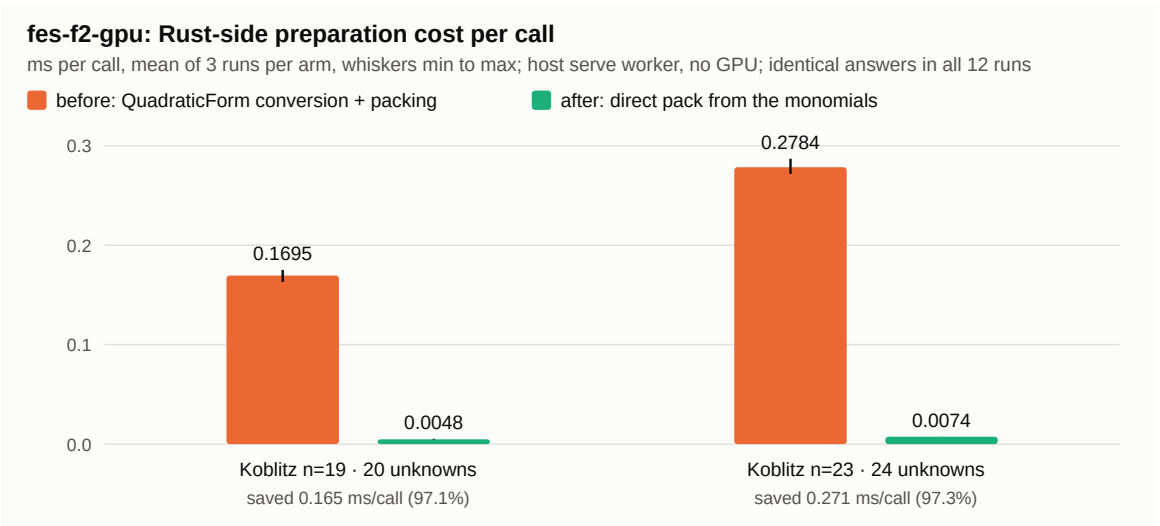


Figure 1: Preparation cost per call, before and after.

Answers unchanged. Every run's result JSON was compared with the first before-run, ignoring wall-clock fields, the per-run machine calibration and the binary hash, with floats compared to 1e-9 relative and everything else exactly: 0 of 6 runs differ at either instance (`raw/identity.txt`). Solver calls (786, 1878) and points per call (2.19, 1.96) are the same in every run. A unit test checks the packed words and both charges against the form path on 90 random systems; the 183 `ic_framework` tests pass with the host serve worker.

Second A/B: a Modal A100

Same two binaries and the same sweeps (`koblitz-n23-gpu.json`, 24 unknowns, 1878 calls; `char2-n25-gpu.json`, 26 unknowns, 2338 calls), now with the CUDA serve worker `gpu/fes/fes_cuda` on an NVIDIA A100-SXM4-80GB (driver 610.57.04, CUDA 12.8.1 image, `sm_80`), built in the container; `FES_SERVE_TIMING=1` adds the worker's own phase timings. Before the A/B the container checked the CUDA worker against the host worker on a two-request smoke test (identical replies). Three runs per arm and instance, arm order alternating (`job_ab.sh`, `modal_ab.py`, `analyze_gpu.py`). Whole-call time is the sum of the `fes-f2-gpu` solver rows' wall time over their solver calls.

instance	whole call, before	whole call, after	saved	prep, before	prep, after
Koblitz n=23, 24 unknowns	0.615 ms	0.389 ms	0.226 ms (36.7%)	0.2201 ms	0.0081 ms
char-2 n=25, 26 unknowns	0.735 ms	0.475 ms	0.259 ms (35.3%)	0.2720 ms	0.0102 ms

The whole call is about 36 percent faster at both sizes. The pipe round trip is unchanged (0.387 to 0.374 ms at 24 unknowns, 0.454 to 0.458 ms at 26), so the saving is the Rust-side preparation, as on the host.

Where the worker spends its time (after the change, microseconds per call):

instance	wait + read request	qsym	host to device	kernel + sync	device to host	reply
24 unknowns	216	1.7	36	103	25	11
26 unknowns	320	1.8	37	192	26	11

Correction (found after the first version of this report): the first column is not the parse. The worker's timer started after a one-byte peek meant to skip the idle wait, but the previous request's trailing newline is still in the stream, so the peek returned at once and the timer then ran through the wait for the next request (the Rust side's preparation, write and verification of the previous call) as well as the parse. Replaying the same requests through the host worker confirms it: a pre-buffered stream reads in 6.6 µs per call, a stream with a 600 µs gap between requests in 714 µs, and the corrected timer on the real pipe gives 13 to 15 µs with the `scanf` parse and 4.4 µs with a buffered hand parser at 19 unknowns. So the column is mostly idle time and the parse is about 13 µs of a 0.39 ms call; the claim that the parse is larger than the kernel is withdrawn. The worker figures are the running mean the worker printed at its last multiple of 200 calls (it is killed at exit), so they cover the first 1800 of 1878 and the first 2200 of 2338 calls. They were taken on the A100 with the uncorrected timer and were not repeated.

What is left of the round trip once the worker's own phases (qsym, copies, kernel, reply: about 0.18 ms at 24 unknowns) are subtracted is about 0.2 ms that no phase timer has located. It is pipe wake-up and stdio latency in both directions plus the parse; [pcc:unverified] that split has not been measured on the GPU.

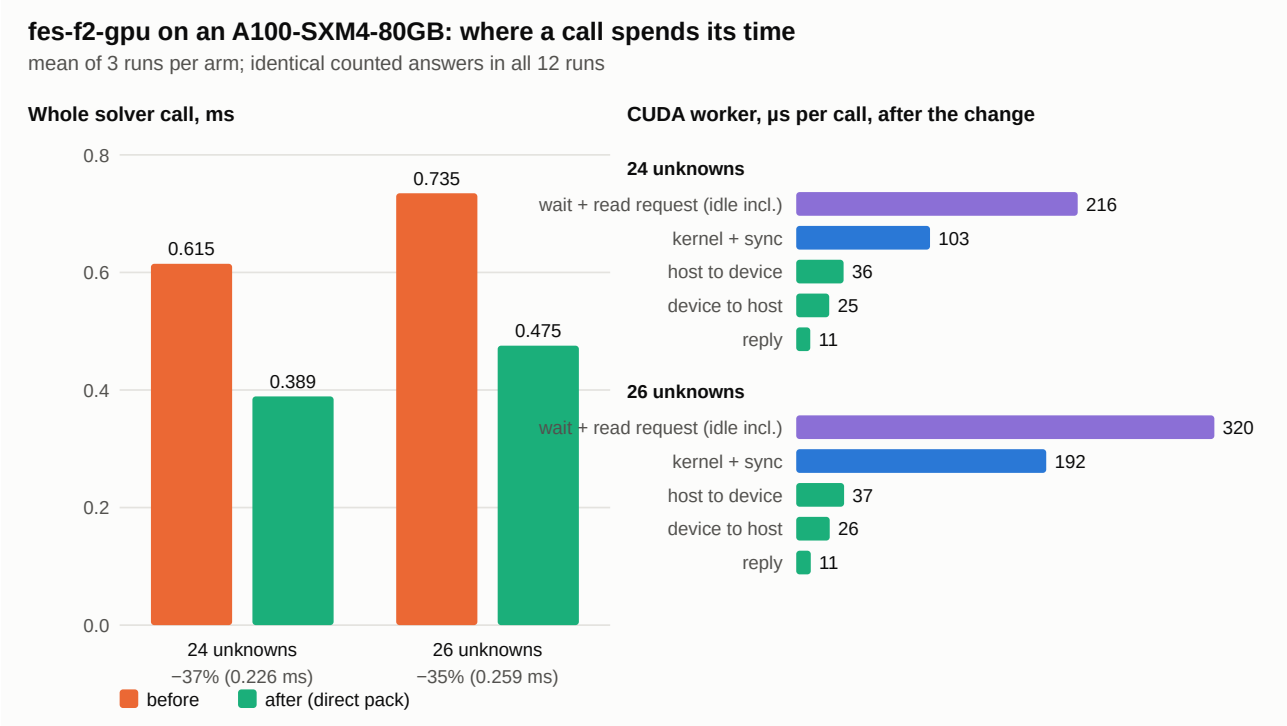


Figure 2: Whole-call time before and after (left) and the CUDA worker's time per call by phase after the change (right), A100.

Answers unchanged. On the counted fields (verdicts, solver calls, counted `ops` and `bitop_word_ops`, hit rates, relations, verified flags) 0 of 6 runs differ from the first before-run at either instance (`raw/identity_a100.txt`). The priced fields (`gae`, `s`, `s_over_rho`, `total_gae`) are counted ops times a per-run machine calibration; at 26 unknowns they differ from the first before-run in 2 of 3 before-arm runs and in 3 of 3 after-arm runs, so they move between runs of the same binary and cannot show a difference between arms (at 24 unknowns they are identical in all runs).

What this does not show

- **The A100 run is a different machine from the earlier end-to-end report's.** Its per-call figures (1.0 ms at 24 unknowns) came from a Runpod A100 with another CPU; the whole-call times here are not comparable with them.
- **No counted-cost effect.** S, S / rho and the gap to the matched rho do not move; this is not an improvement in the repository's metric.
- One authoring host for the CPU runs and one A100 (SXM4-80GB, Modal) for the GPU runs, three runs per arm, no independent replay. The A100 job ran once; its first attempt crashed at container startup (a path bug in the launcher) before executing anything, and was resubmitted as the same single run.

Next

The next cut is not the parse (about 13 µs; a hand parser would save roughly 9 µs of 389, about 2 percent, so it was dropped). The unlocated 0.2 ms of the round trip is the candidate: a persistent worker that is woken by a pipe write, replies by a pipe write, and is read by the Rust side through a line reader. Splitting it needs timestamps on both sides of the pipe on the GPU host, which needs another Modal run.

Files

`run_ab.sh`, `analyze.py`, `results.json`, `job_ab.sh`, `modal_ab.py`, `analyze_gpu.py`, `results_a100.json`, `raw/timings_a100.tsv`, `raw/identity_a100.txt`, `raw/timings.tsv`, `raw/identity.txt`, `raw/binaries.sha256`, sweeps `koblitz-n19-gpu.json`, `koblitz-n23-gpu.json` and `char2-n25-gpu.json`.